

Seventh International Conference on Formal Methods in Computer-Aided Design
**Industrial Strength SAT-based Alignability Algorithm
for Hardware Equivalence Verification**

Daher Kaiss, Marcelo Skaba, Ziyad Hanna, Zurab Khasidashvili
Intel, Israel Design Center, Haifa
{dkaiss,smarcelo,zhanna,zurabk}@iil.intel.com

Abstract

Automatic synchronization (or reset) of sequential synchronous circuits is considered one of the most challenging tasks in the domain of formal sequential equivalence verification of hardware designs. Earlier attempts were based on Binary Decision Diagrams (BDDs) or classical reachability analysis, which by nature suffer from capacity limitations. A previous attempt to attack this problem using non-BDD based techniques was essentially a collection of heuristics aimed at toggling of the latches, and it is not guaranteed that a synchronization sequence will be computed if it exists.

In this paper we present a novel approach for computing reset sequences (and reset states) in order to perform sequential hardware equivalence verification between circuit models. This approach is based on the dual-rail modeling of circuits and utilizes efficient SAT-based engines for Bounded Model Checking (BMC). It is implemented in Intel's sequential verification tool, Seqver, and has been proven to be highly successful in proving the equivalence of complex industrial designs. The synchronization method described in this paper can be used in many other CAD applications, including formal property verification, automatic test generation, and power estimation.

1. Introduction

In this work, we are concerned with *Formal Equivalence Verification (FEV)* of hardware designs. The goal is to verify "functional equivalence" between the specification (e.g., RTL) and the implementation (e.g., schematics) circuit models.

Most of the current generation FEV tools are limited by the requirement that the RTL be state matching with the schematics. That is, there must be a one-to-one correspondence between the state elements of the two compared designs. Therefore, a propositional tautology-checking procedure (e.g., a BDD [3] or SAT [12]-based solver), can be

employed to decide functional equivalence. This form of equivalence verification is known as *combinational equivalence* verification, as opposed to *sequential equivalence* where propositional decision procedures do not suffice and one needs other techniques to perform equivalence verification.

Combinational equivalence verification forces the designers to write detailed RTL in order to meet the state matching requirement. The low level of abstraction in the RTL model has a negative impact on the quality of the RTL [17]. Sequential equivalence verification comes to enable higher abstraction level of the RTL, and to reduce the effort of mapping of sequential elements during FEV.

While many algorithms and tools for several forms of sequential equivalence verification for circuits have existed for more than a decade [28, 2, 32, 36, 15, 19, 20], the industry has started to move towards full-scale sequential equivalence verification only recently [21, 26, 17, 22, 27]. The dominating concepts for sequential equivalence used in the industry all center around alignability equivalence [28]. Alignability equivalence verification can be performed in a compositional manner, where the equivalence of the compared circuits is derived from the equivalence of their corresponding sub-circuits [21, 22, 27]. Therefore, in this work we focus on alignability verification (or more precisely, on strong alignability [17] verification) of sub-circuits, possibly constrained with input properties, and do not discuss the compositionality related topics any further.

Both the BDD-based and SAT-based model checking algorithms require a set of initial states [10, 11, 37, 6, 7, 25, 1, 35]. However, in practice, an initial (or reset) state of a hardware design is not always available. In such cases, an automated technique for computing a reset state for hardware model checking would be an advantage. For alignability equivalence verification, both BDD-based and SAT-based methods have been proposed in the literature [28, 33, 21, 17, 27]. In some of the BDD-based methods [28, 27], computation of reset states of (sub-)circuits is not necessary, while in the SAT-based (strong) alignability verification method one first tries to synchronize the compared (sub-

circuits and then to compare the reset states for state equivalence [33, 21, 17]. The experimental data reported in the above works show that the SAT-based methods scale far better than the BDD-based methods; thus the importance of efficient SAT-based synchronization techniques is evident. Computation of initializing or reset sequences is a difficult problem [30, 33, 8, 29, 24, 18, 9]. Most of the previous algorithms are BDD-based. In [33], a SAT-based method for computing reset states is proposed. That method is essentially a set of heuristics aimed at toggling the latch control signals, as well as toggling some of the inputs and outputs. Toggling these signals is achieved by creating conditions (rather, invariant or safety properties) whose counter-examples can serve as input-vector sequences that guarantee that latch control signals will be asserted and input values will be propagated through the latches, and that some "important" inputs and outputs will toggle. As a result, there is a "good chance" that latches will be initialized. A Bounded Model Checking (BMC [1]) engine was used to generate the counter-examples, as for many sub-circuits the BDDs are not enough even to build the model-checking instances (the next-state functions and initial state relation).

The four-stage initialization algorithm in [33] is not sound in the presence of constraints on (sub-)circuit inputs and internal constraints, due to the unrestricted, random simulation at the first stage of initialization and "partial satisfaction of constraints" in the second stage. Moreover, this method is incomplete in that there is no guarantee (even theoretically) that an initial state or a reset state will be computed if one exists.

In this work we enhance the approach taken in [33] and present the first sound and complete SAT-based technique for computing a reset state. Unlike [33], we use the dual-rail modeling of circuits [4, 5], which allows us to more conveniently and precisely formulate the invariant properties whose counter-examples serve as (parts of the) synchronizing sequences that we are computing. We use the self-alignability ideas from [33] as a complementary method to 3-valued based initialization, and describe a hybrid automatic synchronization algorithm which is more efficient than the self-alignability technique alone and also succeeds in the cases where the 3-valued initialization alone fails.

This paper is organized as follows: In the next section we recall some basic definitions used in this work. In Section 3, we describe the dual-rail modeling of the circuits. Section 4 describes our algorithm for computing synchronization sequences and performing strong alignability verification. Experimental results are discussed in Section 5. Conclusions appear in Section 6.

2. Preliminaries

A circuit design is modeled at the gate level in terms of combinational elements and storage elements. For simplicity, we assume that a storage element is a device that transfers its input to its output when a clock signal is high, and holds the output value when the clock signal is low. A *state* s of a circuit C is any one of the 2^n assignments of boolean values to the n storage elements of C .

Without restricting generality, we assume that any circuit C has exactly one output, o , and a set of n storage elements $L_1 \cdots L_n$. C_1 and C_2 denote the specification and implementation circuits with outputs o_1 and o_2 , respectively. C_1 and C_2 have the same set of inputs (dummy inputs can be added, if necessary). $C_{\Leftrightarrow}$ denotes the combined circuit of C_1 and C_2 – the *product machine* [13] with shared inputs and $o = o_1 \Leftrightarrow o_2$ as the output.

We consider *ternary* modeling of circuit node values. The values can be one of the two *binary* values – T or F, or an *undefined* value – $\perp$ (elsewhere also denoted by X). Given a binary input vector sequence π , $n(C, s, \pi)$ denotes the value of node n of a circuit C after 3-valued simulation of C with π , starting at state s . Similarly, $C(s, \pi)$ denotes the (ternary) state into which π brings C , from state s . The *unknown state* of C is the state in which all the storage elements have the undefined value X. A *binary state* of a circuit C is a state in which all the state elements have binary values.

2.1. Alignability theory

Alignability equivalence [28] is a widely used concept of hardware equivalence verification. Its introduction was motivated by the fact that a power-up state of a hardware design cannot be predicted or controlled, therefore the design must be brought into a smaller set of states where the design is supposed to work correctly.

In this section we recall concepts related to alignability equivalence.

Definition 2.1 *An initializing sequence of C is a sequence of binary inputs which, when applied to the unknown state X of C , brings C into a binary state.*

Definition 2.2 *State (s_1, s_2) of the combined circuit $C_{\Leftrightarrow}$ is an equivalent state (denoted by $s_1 \simeq s_2$) if for any input sequence π , $o_1(C_1, s_1, \pi) = o_2(C_2, s_2, \pi)$. States s_1 and s_2 are then called equivalent states of C_1 and C_2 .*

Definition 2.3 ([28])

1. A binary input sequence π is an *aligning sequence* for a combined state (s_1, s_2) of $C_{\Leftrightarrow}$ if it brings $C_{\Leftrightarrow}$ from state (s_1, s_2) into an equivalent state.
2. Circuits C_1 and C_2 are *alignable*, written $C_1 \cong_{aln} C_2$, if every state of $C_{\Leftrightarrow}$ has an aligning sequence.

It is shown in [28] that $C_1 \cong_{aln} C_2$ iff there is a sequence, called a *universal aligning sequence*, that aligns any state of $C_{\Leftrightarrow}$. When the two circuits coincide $C_1 = C_2 = C$, then following [33] we will speak of the self-alignability of C . Note that C is self-alignable iff it is *weakly synchronizable*, as defined in [31]:

Definition 2.4 A *weak synchronizing sequence* (ws-sequence for short) of a circuit C is an input vector sequence that brings C from any binary state to a subset of states S , called ws-states of C , such that all states of S are equivalent.

Theorem 2.5 (Alignment Theorem [28]) Circuits C_1 and C_2 are alignable if and only if each circuit is weakly synchronizable and there is an equivalent pair $s_1 \simeq s_2$ of states in C_1 and C_2 . The concatenation of ws-sequences of C_1 and C_2 is a ws-sequence for both of them and it weakly synchronizes C_1 and C_2 into equivalent ws-states (when C_1 and C_2 are alignable).

We now recall a more restrictive version of alignability, called *strong-alignability* [17].

Definition 2.6 [23] A *reset or synchronizing sequence* π_r brings C from any binary state to a same state s_r , called a *reset or synchronizing state*.

Note that the set of ws-states and the set of synchronizing states are closed under state transition. Every synchronizing sequence is also a weakly synchronizing sequence. Therefore one can define strong-alignability as:

Definition 2.7 [17] Circuits C_1 and C_2 are *strongly alignable* (denoted by $C_1 \cong_{saln} C_2$) if each circuit is synchronizable and there is an equivalent pair (s_1, s_2) of states in C_1 and C_2 .

The concatenation of synchronizing sequences of C_1 and C_2 is a synchronizing sequence for both of them and it synchronizes C_1 and C_2 into equivalent states (when C_1 and C_2 are strongly alignable).

Strong alignability equivalence can be expressed in the following manner. Let circuit $C_{\Leftrightarrow}$ with an output o be the product of two circuits C_1 and C_2 , and assume that $\mathcal{S}$ is the set of states of $C_{\Leftrightarrow}$; then:

$$C_1 \cong_{saln} C_2 \Leftrightarrow \exists \pi. \forall s \in \mathcal{S}. \left(s_r = C_{\Leftrightarrow}(s, \pi) \wedge \forall \gamma. o(C_{\Leftrightarrow}, s_r, \gamma) = \mathbf{T} \right) \quad (1)$$

The strong alignability verification of two (sub-)circuits is done in two stages: (1) Compute a sequence which guarantees that when applied to a circuit $C_{\Leftrightarrow}$, it brings it to the same state. (2) Perform equivalence verification from the state(s) of the first stage. If the verification passes, then the models are declared (strongly) alignable.

The reasons for choosing strong alignability instead of the regular alignability for performing equivalence verification are driven by practical considerations. Assume that the second stage fails with a counter example. This failure can stem from two possible causes: (1) there is no sequence that aligns the two (sub-)circuits, or (2) this is a real counter example that needs to be debugged. Since it is not practical to require designers to debug counter examples which later on are root caused to a problematic initialization, we decided to work with strong alignability concept. For more methodological discussions concerning the usage of strong alignability, we refer the reader to [17].

3. Dual-rail modeling of hardware designs

In this section, we demonstrate on simple examples how Bounded Model checking (BMC) problems for hardware designs can be encoded using dual-rail modeling [4].

A circuit C is usually represented by a collection of *next-state functions* (NSFs) of the latches, by initial-state constraints on the latches, and by output functions. A NSF is a function of current and next-state values of inputs and latches. For example, consider the circuit C illustrated in Figure 1. It consists of four inputs a, b, c and clk , one latch, l , and the output of the circuit o . As usual, we denote the next-state version of a variable v by v' . So the function of the output o is $l \vee c$, while the NSF of the latch l is $(clk' \wedge a' \wedge b') \vee (\neg clk' \wedge l)$. Available convenient representations for next state functions can be BDDs or boolean expressions. The latter is a simple data structure for representing propositional logic formula. We adopted boolean expressions in our work since uniqueness of BDDs is not needed.

Modeling of sequential logic is done using a *compact* representation of infinite variable sequences. For a signal v , an infinite sequence of propositional variables $\{v_0, v_1, v_2, \dots\}$ represents symbolically its sequential behavior. This allows one to reduce sequential verification problems to propositional satisfiability. The sequence representations can be unrolled to a desired depth k , producing k propositional variables $\{v_0, v_1, \dots, v_{k-1}\}$, which represent all the possible first k values of the signal v . This representation is suitable not only for modeling sequential behavior of inputs, but also for internal combinational signals, storage elements, and outputs. For example, for a given output o in Figure 1, the sequential behavior is represented by a disjunction of the sequences representing l and c . Similarly, we can define the behavior of any storage element by using NSFs.

Dual-rail modeling of circuits was introduced in [4, 5] to enable a more precise modeling of circuit operation, and to enable representation of all *ternary* values with BDDs via a binary encoding. Our adoption of dual rail modeling was

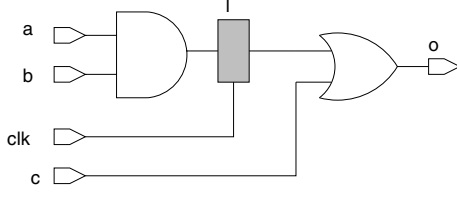


Figure 1: Example of latch and output functions

critical as our SAT solver is based on binary values (T, F) while our algorithm needs ternary values $(T, F, \perp)$. We refer to [34, 16] for more information on dual-rail symbolic simulation.

Each ternary value v is encoded as a pair of binary values (v_h, v_l) , called the *high* and the *low* values. To avoid any confusion, we use F^{dr} , T^{dr} , and $\perp^{dr}$ to denote the dual-rail encoding of T , F and $\perp$, respectively. And $v^{dr} = (v_h, v_l)$ will denote the dual-rail encoding of a ternary variable v . The undefined value $\perp^{dr}$ is encoded as a pair $\perp^{dr} = (T, T)$. The truth constants are encoded by $T^{dr} = (T, F)$ and $F^{dr} = (F, T)$. The pair $\top^{dr} = (F, F)$ encodes a *contaminated* or *over-specified* value.

In order to model sequential logic in dual-rail, it is enough to have dual-rail rules for the combinational operators and the *next state variable operator*, denoted by $'$. The dual-rail rules are as follows: Let $x^{dr} = (x_h, x_l)$ and $y^{dr} = (y_h, y_l)$ be dual-rail encoding of ternary variables x and y . Then

- $(x_h, x_l) \wedge (y_h, y_l) = (x_h \wedge y_h, x_l \vee y_l)$;
- $(x_h, x_l) \vee (y_h, y_l) = (x_h \vee y_h, x_l \wedge y_l)$;
- $\neg(x_h, x_l) = (x_l, x_h)$;
- $(x_h, x_l)' = (x'_h, x'_l)$.

Example 3.1 Let us compute $x^{dr} \oplus T^{dr}$ ($\oplus$ stands for XOR)

$$\begin{aligned}
 x^{dr} \oplus T^{dr} &= ((x_h, x_l) \wedge \neg(T, F)) \vee (\neg(x_h, x_l) \wedge (T, F)) \\
 &= ((x_h, x_l) \wedge (F, T)) \vee ((x_l, x_h) \wedge (T, F)) \\
 &= (x_h \wedge F, x_l \vee T) \vee (x_l \wedge T, x_h \vee F) \\
 &= (F, T) \vee (x_l, x_h) \\
 &= (F \vee x_l, T \wedge x_h) \\
 &= (x_l, x_h) \\
 &= \neg x^{dr}
 \end{aligned}$$

Below we omit the superscript dr in dual-rail formulas.

To ensure that the inputs are always binary in a sequential instance, one needs to add, for any input variable i , a constraint $i_h = \neg i_l$. This guarantees that we do not introduce (F, F) values in the instance. Further, if (F, F) values are not introduced in constraints or in the properties, the NSF's cannot introduce them either (because the above

four operations cannot result in an (F, F) value if the arguments are not over-constrained). Thus, for example, over-constrained values should not appear in a satisfying assignment found by a SAT-solver. An appearance of (F, F) in a satisfying assignment indicates a bug, which is why we do not add a constraint to the instance forbidding over-constrained values on all variables.

A dual-rail NSF is a pair of NSF's of the high and low values. For example, the NSF of the latch in Figure 1 is represented in the dual rail encoding as follows: $l'_h = (clk'_h \wedge a'_h \wedge b'_h) \vee (\neg clk'_h \wedge l_h) = (clk'_h \wedge a'_h \wedge b'_h) \vee (clk'_l \wedge l_h)$ and $l'_l = (clk'_l \vee a'_l \vee b'_l) \wedge (clk'_h \vee l_l)$.

4. Performing strong alignability verification

Given two circuits (C_1, C_2) (specification and implementation circuits), SAT-based strong alignability verification is performed in two stages: (1) compute the reset sequence for both circuits (which by itself produces the reset states) and (2) perform SAT-based state-equivalence verification from the computed reset states. In both stages we assume that the two circuits are modeled in one dual-rail model-checking instance as described in Section 3, and that the instance includes the *constraints* of the two circuits but not the property to be model-checked. The properties will be defined separately depending on the stages of the verification. Since the concatenation of reset sequences for C_1 and for C_2 is a reset sequence for both of them, for simplicity we will describe an algorithm for computing a reset state for one given circuit C .

4.1. The basic algorithm for computing a reset state

Assume a circuit C with exactly one output, o , with n inputs $i_1, \dots, i_n$, and m storage elements $\mathcal{L} = \{L_1 \dots L_m\}$. We will denote the value of a variable v (input, storage element or output) at time-frame k in the unrolled instance by $v[k]$. Computing a reset state consists of two stages: (a) initialization and (b) self-alignability.

(a) Initialization

In this stage, we try to find an initializing sequence γ_1 for the circuit C . If this stage succeeds then there is no need to do the self-alignability stage, and we can proceed directly to the verification stage.

Recall the definition of an *initializing sequence* and the *unknown state* in Section 2. Every initializing sequence is a synchronizing sequence but the inverse is not true. This is true due to the monotonicity of the symbolic simulator, as any binary values resulting when simulating patterns containing X's would also result when the X's are replaced by any combination of 0's and 1's. [5]. An initializing sequence assumes the

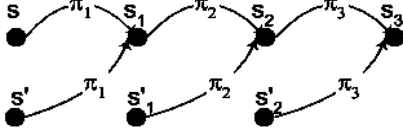


Figure 2: Performing self alignability

unknown state as the starting state (all the storage elements are initialized with the X value). Therefore, for every storage element L modeled by (L_h, L_l) in the dual-rail modeling, we add to the model-checking instance initial-state constraints stating that $L_h[0] = \text{T}$ and $L_l[0] = \text{T}$, which means that we are initializing the state element L with $X = (\text{T}, \text{T})$. We are left with adding to the instance a property such that, if falsified at time-frame k , its counter-example satisfies the following formula:

$$\bigwedge_{L=(L_h, L_l) \in \mathcal{L}} (L_h[k] = \neg L_l[k]) \quad (2)$$

That is, if the counter-example is applied to the circuit C at the starting state X , all the latches will receive binary dual-rail values at the end of the simulation.

To check whether or not the property is falsifiable, one needs to choose a sufficiently large k (the diameter of C) or use other SAT-based methods for full proofs, like the induction method [35] (since BDD-based model checking methods do not scale). In practice, however, SAT-based full-proof methods are computationally expensive compared to the regular BMC and we do not use them as a default. Instead, we chose a reasonably large bound k according to a heuristic and apply BMC till that bound. Furthermore, we modify the above property so that even if full initialization of C in k time steps is not possible, the BMC engine reports a counter-example that initializes a "large number" of latches. We then move to the self-alignability stage of our synchronization algorithm.

(b) Self-alignability

This stage attempts to complete the process of stage (a) by finding a synchronizing sequence $\gamma_1 \gamma_2$ for the circuit C . The computation is done iteratively: each new input sequence π_i is concatenated to the sequence already computed. We start from the state which was obtained after the initialization stage, so that part of the storage elements are not initialized yet. Assume that after applying the sequence γ_1 of the initialization stage, out of the m storage elements in C , we have m' storage elements which are not initialized (have (T, T) value). Intuitively, the algorithm iteratively tries

to enumerate possible values for the m' uninitialized storage elements. The algorithm works as follows:

1. Set $i = 1$.
2. Replicate circuit C and produce new circuit C^{rep} in a way that the non-initialized m' storage element set in C (call it $\mathcal{L}' \subseteq \mathcal{L}$) has a different corresponding encoding set in C^{rep} (call it $\mathcal{L}'^{rep}$). That is, for every storage element $L = (L_h, L_l)$ in C , produce a storage element $L^{rep} = (L_h^{rep}, L_l^{rep})$ in C^{rep} . Our target now is to find a synchronizing sequence for the combined circuit built from C and its copy C^{rep} .
3. Let s_{init} be the ternary state of C at the end of the initialization stage (the latches in $\mathcal{L}'$ have X values in s_{init}). Assume that s and s' are states of C and C^{rep} such that the latches initialized during the initialization stage have the same concrete values as in s_{init} , and the remaining latches have binary dual-rail values. Furthermore, we add the following constraint to the model checking instance to state that s and s' are different states:

$$\bigvee_{L=(L_h, L_l) \in \mathcal{L}'} (L_h[0] = \neg L_h^{rep}[0]) \quad (3)$$

In addition, initial-state constraints defining the values of the initialized latches in s and s' are added to the instance.

4. Find a binary sequence π_i such that when applied to C and C^{rep} starting from s and s' , the result is the same binary state (See Figure 2). This is achieved by adding to the instance a property that, if falsifiable at time-frame k , the counter-example satisfies the following formula:

$$\bigwedge_{L=(L_h, L_l) \in \mathcal{L}} \left(\begin{array}{lcl} (L_h[k] & = & L_h^{rep}[k]) \wedge \\ (L_l[k] & = & L_l^{rep}[k]) \wedge \\ (L_h[k] & = & \neg L_l[k]) \end{array} \right) \quad (4)$$

5. If the property is not falsifiable till a bound k , then either the circuit is not synchronizable, or we did not choose a sufficiently large bound. We usually choose a large bound, so most probably the reason for the failure at this stage is due to the fact that the circuit is not synchronizable. The algorithm aborts with failure in this situation.
6. Otherwise, check whether there are two binary states of C , C_{rep} such that the latches that are initialized after the initialization stages have the same values as in s_{init} , and when applying the sequence $\pi_1 \dots \pi_i$ to both of them, the result is

two different binary states, s_i and s'_i . This is done by adding a property that, if falsifiable at bound k , the following formula is satisfied

$$\bigvee_{L=(L_h, L_l) \in \mathcal{L}} \left(\begin{array}{l} (L_h[k] = \neg L_h^{rep}[k]) \vee \\ (L_l[k] = \neg L_l^{rep}[k]) \end{array} \right) \quad (5)$$

and by restricting the behavior of the inputs of the circuit to comply with $\pi_1 \cdots \pi_i$.

7. If such states don't exist, then C is synchronizable and the sequence $\gamma_1 \pi_1 \cdots \pi_i$ is a synchronizing sequence for it.
8. Otherwise, go to step 4 with $s := s_i, s' := s'_i$ (s.t. $s_i = C(\perp, \gamma_1 \pi_1 \cdots \pi_i)$ and $s'_i = C_{rep}(\perp, \gamma_1 \pi_1 \cdots \pi_i)$) and with $i := i + 1$.

The algorithm is sound in the sense that if a sequence is found, it is guaranteed to be a synchronizing sequence. This is due to the iterative process which looks for new states that are not yet joined (i.e., transferred into the same state) by the already computed sequence. Furthermore, by the same argument, we can conclude that if in the last item of stage (b) of the algorithm we define $s_i = C(s_{sym}, \gamma_1 \pi_1 \cdots \pi_i)$ and $s'_i = C_{rep}(s_{sym}, \gamma_1 \pi_1 \cdots \pi_i)$ (rather than $s_i = C(\perp, \gamma_1 \pi_1 \cdots \pi_i)$ and $s'_i = C_{rep}(\perp, \gamma_1 \pi_1 \cdots \pi_i)$), where s_{sym} is a symbolic binary representation of a state in C , the algorithm becomes complete in the sense that if C is synchronizable, the algorithm will build a synchronizing sequence. When this algorithm was applied to real designs, the number of iterations was surprisingly small (most of the time 1 or 2 and always less than 10), which makes the algorithm very practical. This is partly due to the fact that many of the storage elements are already initialized during the initialization stage.

This hybrid system of combining initialization with self-alignability is indeed the breakthrough of our method, since initialization alone is insufficient and running self-alignability alone is not practical due to the large number iterations needed.

4.2. Performing strong alignability verification

Once the reset states of C_1 and C_2 are computed, we mainly use the induction-based algorithms [35] to check that the two reset states are state-equivalent. The induction based algorithms perform reasonably well when a full proof is sought. The instance should contain the NSFs, the initial-state constraints describing the reset states obtained during stage (1), the constraints, and a property stating that $o_{\Leftrightarrow} = T$.

Table 1: Single Outputs

Ckt	Inps	Latches	Result	#Self	Synch. (sec.)	Verify (sec.)
o_1	2283	40	EQUAL	2	300.5	617.1
o_2	396	884	EQUAL	0	2.2	40.9
o_3	111	133	EQUAL	0	0.3	10.5
o_4	969	112	EQUAL	0	2.4	9.6
o_5	96	132	EQUAL	1	1.3	8.7
o_6	277	873	EQUAL	0	1.6	2.9
o_7	381	402	EQUAL	2	0.1	0.8
o_8	398	984	EQUAL	0	2.4	0.2
o_9	275	748	EQUAL	0	0.8	0.1
o_{10}	23	72	Not Init.	2	0.9	0

Table 2: Total Run Times

Ckt	Gates	Latches	Outs	Inps	CPU(sec.)
C_1	362604	12693	389	397	4700
C_2	423070	23913	349	302	5722
C_3	157110	22080	572	578	2478

5. Results

The algorithms described are implemented in Intel's Sequential Equivalence VERifier, Seqver [17]. Seqver is being used to verify the next generation of Intel's processors, and has found many bugs. Tables 1 and 2 contain data about the size of the problems that were verified and the run time it took the tool to verify them. The run time was measured on a 2.4GHz Linux machine with 2GB memory. Table 1 illustrates the performance of the algorithm running on a single output. Note that for some of the outputs, initialization was not enough and we had to perform self-alignability (#self is the number of self-alignability iterations). The run time results show the run time for the synchronization and the verification stages. The number of iterations in the self-alignability stage is 1-2 on the average. Table 2 illustrates total run time for Seqver on representative circuits. To the best of our knowledge, no previous work has reported a tool that is able to perform (compositional) alignability verification on such large (sub-)circuits, and in such a short time, for circuits without predefined reset states.

6. Summary

We have given a description of a SAT-based algorithm for performing efficient alignability equivalence verification. This algorithm outperforms previous BDD based algorithms (which suffer from capacity limitations), and completes previous SAT-based attempts.

Acknowledgments We thank Gabriel Bischoff, Dinos Moundanos, Joonyoung Kim and Abhijit Jas for careful reading and valuable inputs.

References

- [1] A. Biere, A. Cimatti, E. Clarke. *Symbolic model checking without BDDs*, Tools and Algorithms for the Construction and Analysis of Systems, TACAS 1999.
- [2] G. P. Bischoff, K. S. Brace, S. Jain, R. Razdan. Formal implementation verification of the bus interface unit for the Alpha 21164 microprocessor, IEEE/ACM International Conference on Computer Design, ICCD 1997.
- [3] R.E. Bryant Graph-based algorithms for Boolean function manipulation, IEEE Trans.Computers, C-35(8), 1986.
- [4] R. E. Bryant. *Boolean Analysis of MOS Circuits*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, Vol. CAD-6, No. 4, 1987
- [5] R.E. Bryant, C.-J.H. Seger. *Digital circuit verification using partially-ordered state models*, Twenty-Fourth International Symposium on Multiple-Valued Logic, 1994.
- [6] G. Cabodi, P. Camurati, S. Quer. *Improved reachability analysis of large finite state machines*, ICCAD 1996.
- [7] G. Cabodi, P. Camurati. *Symbolic FSM traversals based on the transition relation*, IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems, vol 16, n. 5, 1997.
- [8] H. Cho, S.-W. Jeong, F. Somenzi, C. Pixley. *Synchronizing sequences and symbolic traversal techniques in test generation*, J. Electron. Test.: Theory & Applications, vol. 4, n. 2, 1993.
- [9] F. Corno, P. Prinetto, M. Rebaudengo, M. Sonza Reorda and G. Squillero. *A new approach for initialization sequences computation for synchronous sequential circuits*, IEEE VLSI in Computers and Processors, 1997.
- [10] O. Coudert, C. Berthet, J.C. Madre. *Verification of synchronous sequential machines based on symbolic execution*, Workshop of Automatic Verification Methods for Finite State Systems, 1989.
- [11] O. Coudert, J.C. Madre. *A Unified framework for the formal verification of sequential circuits.*, ICCAD 1990.
- [12] Davis,M., G.Logemann, D. Loveland, A machine program for theorem-proving, CACM 5(7), 1962.
- [13] G.D. Hachtel, F. Somenzi. *Logic Synthesis and Verification Algorithms*, Kluwer Academic Publishers, 1998.
- [14] J. E. Hopcroft, J. D. Ullman. *Introduction to Automata Theory, Languages and Computation*. Reading, MA: Addison-Wesley, 1979.
- [15] S.-Y. Huang, K.-T. Cheng, K.-C. Chen. *Verifying sequential equivalence using ATPG techniques*, ACM Transactions on Design Automation of Electronic Systems, 2001.
- [16] R.B. Jones. *Symbolic Simulation Methods for Industrial Formal Verification*, Kluwer Academic Publishers, 2002.
- [17] D. Kaiss, S. Goldenberg, Z. Hanna, and Z. Khasidashvili. *Seqver: A Sequential Equivalence Verifier for Hardware Designs*, IEEE International Conference on Computer Design, ICCD 2006.
- [18] M. Keim, B. Becker and B. Stenner. *On the (non)-resetability of synchronous sequential circuits*, IEEE VLSI Test Symposium, 1996.
- [19] Z. Khasidashvili, Z. Hanna, TRANS: Efficient sequential verification of loop-free circuits, HLDVT 2002.
- [20] Z. Khasidashvili, and Z. Hanna. *SAT-Based methods for sequential hardware equivalence verification without synchronization*, BMC'03, ENTCS 89 (4), 2003.
- [21] Z. Khasidashvili, M. Skaba, D. Kaiss and Z. Hanna. *Theoretical Framework for Compositional Sequential Hardware Equivalence Verification in Presence of Design Constraints*, IEEE International Conference on Computer Aided Design, ICCAD 2004.
- [22] Z. Khasidashvili, M. Skaba, D. Kaiss and Z. Hanna. *Post-reboot equivalence and compositional verification of hardware*, FMCAD 2006.
- [23] Z. Kohavi. *Switching and Finite Automata Theory*, McGraw-Hill, 1978.
- [24] Y. Lu and I. Pomeranz. *Synchronization of large sequential circuits by partial reset*, IEEE VLSI Test Symposium, 1996.
- [25] K.L. McMillan. *Symbolic Model Checking: An Approach to the State Explosion Problem*, Kluwer Academic Publishers, 1993.
- [26] H. Mony, J. Baumgartner, V. Paruthi, R. Kanzelman. *Exploiting suspected redundancy without proving it*, Design Automation Conference, DAC 2005.
- [27] I-H. Moon, P. Bjesse, C. Pixley. A compositional approach to the combination of combinational and sequential equivalence checking of circuits without known reset states, DATE 2007.
- [28] C. Pixley. *A theory and implementation of sequential hardware equivalence*, IEEE transactions on CAD, 1992.
- [29] C. Pixley, and G. Beihl. *Calculating resettability and reset sequences*, International Conference on Computer-Aided Design, ICCAD 1991.
- [30] C. Pixley, S.-W. Jeong, G.D. Hachtel. *Exact calculation of synchronizing sequences based on binary decision diagrams*, IEEE transactions on Computer-Aided Design, vol. 13, 1994.
- [31] I. Pomeranz, S. M. Reddy. *On removing redundancies from synchronous sequential circuits with synchronizing sequences*, IEEE Trans. Comput., pp.20-32, 1996.
- [32] Ranjan R.K., V. Singhal, F. Somenzi, R.K. Brayton, Combinational verification for sequential circuits, DATE 1999.
- [33] A. Rosenmann and Z. Hanna. *Alignability equivalence of synchronous sequential circuits*, HLDVT, 2002.
- [34] C.-J. H. Seger, and R. E. Bryant. *Formal verification by symbolic evaluation of partially-ordered trajectories*, Formal Methods in System Design, vol. 6, no. 2, 1995.
- [35] M. Sheeran, S. Singh, G. Stålmarck. *Checking safety properties using induction and a SAT-solver*, FMCAD, 2000.
- [36] V. Singhal, C. Pixley, A. Aziz, and R.K. Brayton, Theory of safe replacement for sequential circuits, IEEE Trans. on CAD of integrated circuits and systems, vol. 20, n.2, 2001.
- [37] H. Touati, H. Savoj, B. Lin, R.K. Brayton, A. Sangiovanni-Vincentelli. *Implicit enumeration of finite state machines using BDDs*, CAD'90, 1990.